МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ИМЕНИ М. В. ЛОМОНОСОВА
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ МАТЕМАТИКИ И КИБЕРНЕТИКИ

Компьютерный практикум по курсу

“Введение в численные методы”

Задание номер два

Численные методы решения дифференциальных уравнений

ОТЧЕТ

о выполненном задании

студента 204 учебной группы факультета ВМК МГУ

Лютова Владимира

Москва, 2014

Оглавление

Практическая работа №2....3
Лицензия....3
Подвариант 1....4
Цель работы....4
Постановка задачи....4
Цели и задачи практической работы....5
Метод Рунге-Кутта....6
Описание программы....7
Исходный текст модулей....7
Тесты....7
Исследование....1
Вывод....2
Подвариант 2....3
Цель работы....3
Постановка задачи....3
Цели и задачи практической работы....3
Описание метода решения....4
Исследование....5
Вывод....6
Приложение....7

Практическая работа №2

Лицензия

ode - ordinary differential equation solver
Copyright (C) 2014 vslutov

This program is free software: you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation, either version 3 of the License, or
(at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License
along with this program. If not, see <http://www.gnu.org/licenses/>.

Подвариант 1

Решение задачи Коши для дифференциального уравнения первого порядка или системы дифференциальных уравнений первого порядка

Цель работы

Освоить методы Рунге-Кутта второго и четвертого порядков точности, применяемые для численного решения задачи Коши для дифференциального уравнения (или системы дифференциальных уравнений) первого порядка

Постановка задачи

Рассматривается обыкновенное дифференциальное уравнение первого порядка, разрешенное относительно производной и имеющее вид:

```
y'(x) = f(x, y)
```

с дополнительным начальным условием, заданным в точке `x = x0`:

```
y(x0) = y0
```

Предполагается, что функция `f = f(x, y)` такова, что гарантирует существование и единственность решения задачи Коши.

В том случае, если рассматривается не одно дифференциальное уравнение, а система обыкновенных дифференциальных уравнений первого порядка, разрешенных относительно производных неизвестных функций, то соответствующая задача Коши имеет вид (на примере двух дифференциальных уравнений):

```
u'(x) = f(x, u, v)
v'(x) = g(x, u, v)
```

Допольнительные начальные условия задаются в точке x0:

```
u(x0) = u0
v(x0) = v0
```

Также предполагается, что функции `f` и `g` заданы так, что это гарантирует существование и единственность решения задачи Коши (3)-(4), но уже для системы обыкновенных дифференциальных уравнений первого порядка в форме, разрешенной относительно производных неизвестных функций.

Заметим, что к подобным задачам сводятся многие важные задачи, возникающие в механике (уравнения движения материальной точки), небесной механике, химической кинетике, гидродинамике и т.п.

Цели и задачи практической работы

- Решить задачу Коши (l)-(2) (или (3)-(4)) наиболее известными и широко используемыми на практике методами Рунге-Кутта второго и четвертого порядка точности, аппроксимировав дифференциальную задачу соответствующей разностной схемой (на равномерной сетке); полученное конечно-разностное уравнение (или уравнения в случае системы), представляющее фактически некоторую рекуррентную формулу, просчитать численно;
- Найти численное решение задачи и построить его график;
- Найденное численное решение сравнить с точным решением дифференциального уравнения (подобрать специальные тесты, где аналитические решения находятся в классе элементарных функций).

Метод Рунге-Кутта

Методы Рунге—Кутта — важное семейство численных алгоритмов решения обыкновенных дифференциальных уравнений и их систем. Данные итеративные методы явного и неявного приближённого вычисления были разработаны около 1900 года немецкими математиками К. Рунге и М. В. Куттой.

Метод Рунге—Кутта четвёртого порядка столь широко распространён, что его часто называют просто методом Рунге—Кутта.

Рассмотрим задачу Коши:

```
y'(x) = f(x, y)
y(x0) = y0
```

Тогда приближенное значение в последующих точках вычисляется по итерационной формуле:

```
y[n+1] = y[n] + step / 6 * (k1 + 2 * k2 + 2 * k3 + k4)
```

Вычисление нового значения проходит в четыре стадии:

```
k1 = f(x[n], y[n])
k2 = f(x[n] + step / 2, y[n] + step * k1 / 2)
k3 = f(x[n] + step / 2, y[n] + step * k2 / 2)
k4 = f(x[n] + step, y[n] + step * k3)
```

где `step` — величина шага сетки по `x`.

Этот метод имеет четвёртый порядок точности, то есть суммарная ошибка на конечном интервале интегрирования имеет порядок `O(step^4)` (ошибка на каждом шаге порядка `O(step^5)`).

Также существует метод Рунге-Кутта второго порядка, который работает аналогично вышеописанному, но выполняет два шага пересчета вместо четырех при расчете каждого значения.

Если мы решаем задачу Коши для системы ОДУ, следует рассматривать искомую функцию `y` и функцию `f(x, y)`, как векторзначные функции.

Описание программы

Программа разбита на модули. Если запустить любой из модулей, кроме тестирующего и основного, то будут выполнены тесты, проверяющие корректность работы функций, определенных в этом модуле.

Список модулей:

- `test.py` - тестирующий модуль, используйте `python test.py` для проверки программы на корректность.
- `grid_helpers.py` - содержит функции, помогающие обрабатывать сеточные функции.
- `generate_images.py` - создает графики для этого отчета.
- `problem.py` - формирует задачу Коши.

Описание всех функций есть в документации по модулям, которую можно получить с помощью системы pydoc, например, выполнив команду `pydoc <имя модуля>`.

Исходный текст модулей

```
Примечание: этот и последующий пункты относятся в равной степени как к
первому подварианту, так и ко второму.
```

Исходный текст модулей доступен по ссылке https://github.com/vslutov/ode.

Лицензия под которой распространяется данная программа и ее исходный код - GNU GPLv3.

Самые значимые отрывки кода есть в **приложении** к этому отчету.

Тесты

В программу включен набор тестов для каждой функции (эти тесты работают с помощью системы doctest и включены прямо в строки документации).

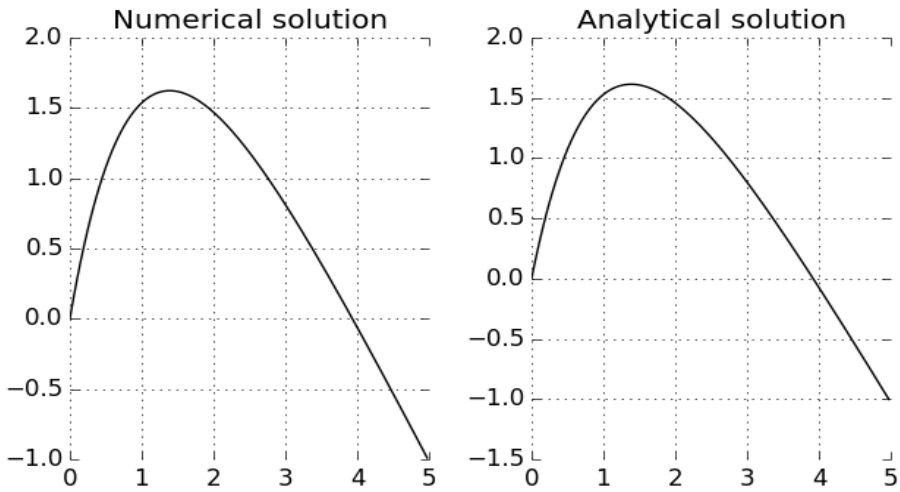
Также есть файл test.py, который запускает помимо уже описаных тестов тесты на работоспособность всей программы в целом. Он работает с использованием системы unittest.

Исследование

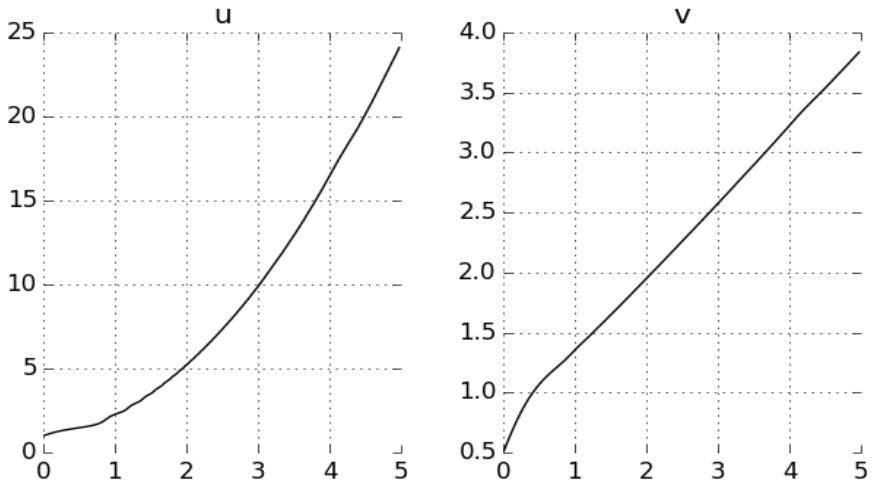
На задачах, допускающих аналитическое решение метод нашел ответ с маленькой погрешностью.

Ниже приведены решения некоторых задач.

```
y'(x) = 3 - y - x
y(0) = 0
```




```
u'(x, u, v) = sin(2 * u^2) + x + v
v'(x, u, v) = x + u - 2 * v^2 + 1
u(0) = 1
v(0) = 0.5
```



Вывод

Мы освоили методы Рунге-Кутта второго и четвертого порядков точности, применяемые для численного решения задачи Коши для дифференциального уравнения (или системы дифференциальных уравнений) первого порядка

Подвариант 2

Решение краевой задачи для обыкновенного дифференциального уравнения второго порядка, разрешенного относительно старшей производной.

Цель работы

Освоить метод прогонки решения краевой задачи для дифференциального уравнения второго порядка.

Постановка задачи

Рассматривается линейное дифференциальное уравнение второго порядка:

```
y''(x) + p(x) * y'(x) + q(x) * y(x) = -f(x)
0 < x < 1
```

С дополнительными условиями в граничных точках:

```
m * y(a) + l * y'(a) = n
p * y(b) + q * y'(b) = r
```

Цели и задачи практической работы

- Решить краевую задачу методом конечных разностей, аппроксимировав ее разностной схемой второго порядка точности (на равномерной сетке); полученную систему конечно-разностных уравнений решить методом прогонки;
- Найти разностное решение задачи и построить его график;
- Найденное разностное решение сравнить с точным решением дифференциального уравнения (подобрать специальные тесты, где аналитические решения находятся в классе элементарных функций)

Описание метода решения

При применении метода конечных разностей к краевым задачам для уравнений второго порядка получается “трехчленная система” линейных алгебраических уравнений, каждое из которых содержит три соседних неизвестных. Для решения такой системы разработан специальный метод, называемый методом прогонки.

Перейдем к конечно разностным уравнениям. Для этого заменим первую и вторую производные их разностным аналогом.

```
y' = (y[i+1] - y[i-1]) / (2 * h)
y'' = (y[i+1] - 2 * y[i] + y[i + 1]) / h^2
```

Получаем линейную систему уравнений

```
A[i] * y[i-1] + B[i] * y[i] + C[i] * y[i+1] = D[i]
```

Ее решаем методом прогонки.

Метод прогонки основывается на предположении, что искомые неизвестные связаны рекуррентным соотношением:

```
y[i] = y[i+1] * a[i+1] + b[i+1]
```

Используя это соотношение выразим `a[i+1]` и `b[i+1]` через коэфициенты матрицы и предыдущие члены. Затем из граничных условий найдем `a[1]` и `b[1]` и построим обе последовательности слева направо.

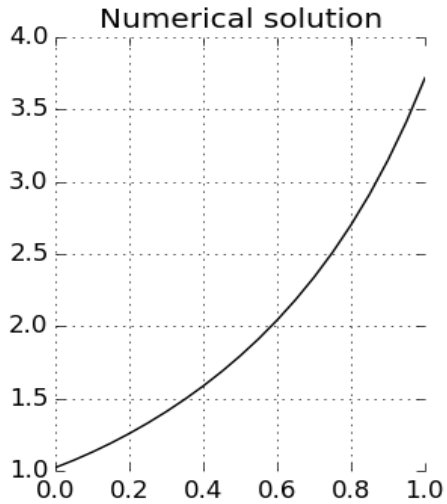
Также найдем `y[n]` из граничных условий и построим всю последовательность `y` справа налево.

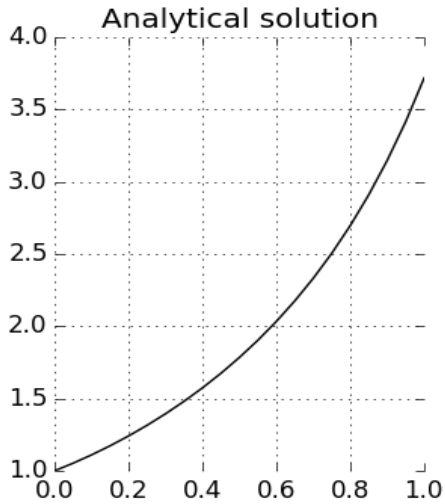
Исследование

На задачах, допускающих аналитическое решение метод нашел ответ с маленькой погрешностью.

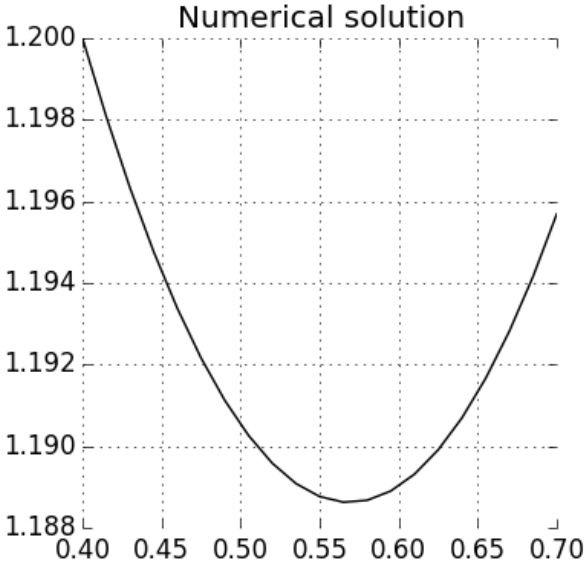
Ниже приведены решения некоторых задач.

```
y'' - 2 * x * y' - 2 * y = - 4 * x
y(0) - y'(0) = 0
y(1) = 1 + e
```






```
y'' - 0.5  * x * y' + y = 2
y(0.4) = 1.2
y(0.7) + 2 * y'(0.7) = 1.4
```



Вывод

Мы научились решать краевую задачу методом конечных разностей, аппроксимируя ее разностной схемой второго порядка точности (на равномерной сетке).

Но ответ получился первой степени точности, так как начальные значения для метода прогонки мы находили из оценки на левую и правые производные функции, а они приближают производную функции с первой степенью точности.

Приложение

Интересные участки кода

```
    def runge_kutta_2(problem, grid):
        diff, init = problem
        func_count = len(init) - 1

        step = grid[1] - grid[0]
        length = len(grid)

        solution = list(range(func_count) for i in xrange(length))
        solution[0] = init[1:]

        for i in xrange(1, length):
            part = diff(grid[i-1], *solution[i-1])
            pred = [solution[i-1][j] + part[j] * step for j in
xrange(func_count)]
            part = diff(grid[i-1], *pred)
            corr = [solution[i-1][j] + part[j] * step for j in
xrange(func_count)]
            solution[i] = [0.5 * (pred[j] + corr[j]) for j in
xrange(func_count)]

        return solution


    def runge_kutta_4(problem, grid):
        diff, init = problem
        func_count = len(init) - 1

        step = grid[1] - grid[0]
        length = len(grid)

        solution = list(range(func_count) for i in xrange(length))
        solution[0] = init[1:]

        for i in xrange(1, length):
            k1 = diff(grid[i-1], *solution[i-1])
            k2 = diff(grid[i-1] + step * 0.5,
                      *(solution[i-1][j] + 0.5 * k1[j] * step
                        for j in xrange(func_count)))
            k3 = diff(grid[i-1] + step * 0.5,
                      *(solution[i-1][j] + 0.5 * k2[j] * step
                        for j in xrange(func_count)))
            k4 = diff(grid[i-1] + step,
                      *(solution[i-1][j] + k3[j] * step
                        for j in xrange(func_count)))
            solution[i] = [solution[i-1][j] +
                           step/6 * (k1[j] + 2 * k2[j] + 2 * k3[j] + k4[j])
                           for j in xrange(func_count)]

        return solution
```

```
def boundary(problem, grid):
    coeff, bound = problem

    step = grid[1] - grid[0]
    length = len(grid)

    a, b = range(length), range(length)

    A, B, C = bound[0]
    a[1] = -B / (A * step - B)
    b[1] = C * step / (A * step - B)

    for i in xrange(1, length - 1):
        co = coeff(grid[i])
        p = 1/step**2 - co[0]/(2*step)
        q = co[1] - 2/step**2
        r = 1/step**2 + co[0]/(2*step)
        s = co[2]

        a[i + 1] = -r / (p * a[i] + q)
        b[i + 1] = (s - p * b[i]) / (p * a[i] + q)

    solution = [[0] for i in xrange(length)]

    A, B, C = bound[1]
    solution[length - 1][0] = ((B * b[length-1] + C * step) /
                               (A * step + B * (1 - a[length-1])))

    for i in xrange(length-1, 0, -1):
        solution[i-1][0] = a[i] * solution[i][0] + b[i]

    return solution
```